CI/CD: pipeline, artifact, release và rollback

Một chủ đề, một tài liệu: mô hình, bài tập, lỗi thường gặp và câu hỏi phỏng vấn.

Đọc theo thứ tự trong tập. Làm lab trên môi trường riêng, ghi kết quả và đối chiếu tài liệu chính thức theo phiên bản bạn đang dùng.

Bám theo bốn JD người học cung cấp | 11/10/2026

Tài liệu độc lập theo một chủ đề. Đọc mô hình trước, làm lab trên môi trường riêng, ghi bằng chứng và tự giải thích failure mode. Các ví dụ dựa trên ứng dụng Notes API và bốn JD đã cung cấp.

Từ mã nguồn đến bản phát hành

Mã nguồn là công thức; artifact là kết quả của quy trình build. Khi một commit được đánh dấu để phát hành, ta muốn xác định chính xác mã nào, dependency nào và quy trình nào đã tạo artifact. Nếu mỗi môi trường tự build lại từ một nhánh đang thay đổi, bạn khó bảo đảm test đã chạy trên đúng thứ đang phục vụ người dùng. Cách dễ kiểm soát hơn là build một lần, định danh artifact, rồi chuyển chính artifact đó qua các môi trường.

Test giúp kiểm tra kỳ vọng, nhưng không chứng minh phần mềm không còn lỗi. Unit test cô lập hàm hoặc module; integration test kiểm tra tương tác giữa thành phần như API và database; smoke test chạy sau triển khai để xác nhận vài luồng quan trọng. Lint và static analysis bắt các vấn đề khác. Một pipeline tốt chọn lớp kiểm tra nhanh cho mỗi commit, lớp tốn kém hơn ở vị trí phù hợp, và hiển thị lỗi đủ rõ để người sửa tìm nguyên nhân.

Ví dụ cụ thể

API ghi chú có endpoint **POST /notes**. Unit test kiểm tra dữ liệu rỗng bị từ chối; integration test tạo ghi chú trong database thử nghiệm rồi đọc lại; smoke test sau deploy gọi health endpoint và một request đọc. Nếu unit test qua nhưng integration test lỗi kết nối, hướng điều tra chuyển sang database/configuration. Nếu tất cả test trước deploy qua nhưng smoke test lỗi 502, hướng điều tra chuyển sang route, proxy hoặc tiến trình vừa triển khai.

Một release có thể thất bại dù pipeline màu xanh: dữ liệu thật lớn hơn dữ liệu test, secret sai, tài nguyên thiếu hoặc dependency ngoài không đáp ứng. Vì thế cần observability và rollback từ sớm. Ghi rõ điều gì được kiểm tra tự động và điều gì chưa.

Lab trên ứng dụng mẫu

Điều kiện: ứng dụng mẫu chạy local. Viết một test cho request hợp lệ và một test cho request sai. Cố ý sửa mã để test thứ hai thất bại; xem thông báo lỗi và sửa lại. Ghi commit chứa test. Nếu chưa biết framework test của ngôn ngữ, dùng hướng dẫn chính thức của framework bạn chọn, không sao chép đoạn cấu hình không hiểu.

CI và đường đi của một commit

Continuous Integration (CI) là cách nhóm hợp nhất thay đổi thường xuyên, kèm build và kiểm tra tự động. Một workflow thường có trigger, job, runner và step. Trigger quyết định khi nào chạy; runner là máy thực hiện; job nhóm các step; artifact là đầu ra được lưu hoặc chuyển tiếp. Hãy bắt đầu bằng pipeline đơn giản: checkout -> cài dependency đã khóa -> lint/test -> build image. Khi pipeline lỗi, xem step đầu tiên thất bại và log liên quan thay vì chạy lại vô hạn.

Runner xử lý mã từ repository và đôi khi có quyền truy cập token. Vì vậy quyền của pipeline phải tối thiểu; secret không được in ra log; workflow nhận đóng góp từ người lạ cần kiểm tra cách xử lý token. Caching có thể tăng tốc nhưng cache không phải nguồn chân lý: xóa cache mà build vẫn phải đúng. Nếu dùng action của bên thứ ba, xem nguồn gốc và cách ghim phiên bản theo khuyến nghị của nền

tảng.

Continuous delivery nghĩa là phần mềm luôn ở trạng thái có thể phát hành theo quy trình đã định. Continuous deployment đi thêm một bước: thay đổi qua kiểm tra được tự động đưa vào môi trường đích. Nhóm có thể cần bước phê duyệt vì lý do sản phẩm hoặc rủi ro; việc đó không làm bài học CI/CD kém giá trị. Mục tiêu là thay đổi nhỏ, có thể quan sát và khôi phục.

Lab CI

Điều kiện: repository thử nghiệm trên nền tảng CI bạn chọn. Viết workflow chạy test của ứng dụng mẫu khi có pull request hoặc push vào nhánh bài học. Sau khi chạy xanh, cố ý tạo một test lỗi, đẩy lên nhánh riêng, xem step và exit code, rồi sửa. Ghi lại commit, đường dẫn run và nguyên nhân. Không đặt secret thật vào repo hoặc nội dung workflow; nếu pipeline chưa cần deploy, nó không cần quyền cloud.

Tự xây một pipeline có bằng chứng

Hãy diễn đạt pipeline thành hợp đồng. Đầu vào: commit cụ thể. Kiểm tra: cú pháp Python, test cho endpoint và tạo image. Đầu ra: log kết quả, artifact image có định danh, và trạng thái pass/fail. Nếu test fail, không phát hành artifact như bản dùng được. Một pipeline chỉ in chữ "success" mà không chạy kiểm tra có ý nghĩa không mang lại nhiều bảo đảm.

Với GitHub Actions, workflow là file YAML trong **.github/workflows/**. **on** xác định sự kiện, **jobs** định nghĩa công việc, **runs-on** chọn runner và **steps** chạy lệnh/action. Bắt đầu bằng workflow chỉ kiểm tra cú pháp và chạy test, chưa cần cloud credential. Workflow mẫu cần phù hợp phiên bản runner và action hiện hành; khi viết bài thực hành, tra tài liệu GitHub để xác nhận cách ghim phiên bản và quyền token.

Lab pipeline theo hai tình huống

Tạo một test kiểm tra **GET /health** trả 200 và version. Đầu tiên chạy test trên máy; ghi lệnh và kết quả. Sau đó đưa cùng lệnh vào CI. Đẩy một commit hợp lệ để pipeline xanh. Tạo nhánh lab và sửa test để kỳ vọng sai; đẩy commit, chụp tên step fail và dòng log chỉ ra assertion. Sửa lại test hoặc code và đẩy tiếp. Lúc này bạn có một chuỗi chứng minh pipeline thực sự chặn lỗi, không chỉ là cấu hình đẹp.

Phân biệt ba loại lỗi trong CI: test assertion fail (logic/kỳ vọng), environment fail (thiếu dependency, khác phiên bản), infrastructure fail (runner hoặc registry không đáp ứng). Cách xử lý khác nhau. Với lỗi môi trường, so sánh phiên bản Python, working directory, biến môi trường và file dependency đã khóa giữa local và runner. Đừng thêm retry trước khi biết lỗi nào đang xảy ra.

Bài tập thiết kế release và rollback

Viết bảng bốn cột cho phiên bản B: thay đổi gì, bằng chứng kiểm tra, tín hiệu sau deploy, cách rollback. Ví dụ: thay endpoint tạo ghi chú -> unit/integration test và smoke POST/GET -> tỷ lệ 5xx, thời gian xử lý, log validation -> triển khai lại image A nếu B lỗi, sau khi xác nhận database vẫn tương thích. Sau đó thêm một thay đổi schema; nêu vì sao rollback image có thể thất bại và cách chia migration thành bước tương thích.

Một release có thể dùng checkpoint 5 phút và 30 phút. Sau 5 phút xem smoke test và lỗi rõ; sau 30 phút xem xu hướng latency/error. Các con số chỉ là ví dụ cho lab, không phải ngưỡng áp dụng mọi

production. Điều quan trọng là điều kiện thành công và dừng được viết ra trước khi triển khai.

Một bản phát hành cần tối thiểu: phiên bản xác định, nội dung thay đổi, điều kiện đưa lên, cách kiểm tra sau triển khai, người chịu trách nhiệm và cách quay về. Rollback bằng image cũ không luôn đủ nếu database schema đã đổi không tương thích. Tập thói quen tách thay đổi schema có thể tương thích ngược, thử migration trên dữ liệu giả và dự tính cách phục hồi.

Cổng năng lực: bạn tự build image, chạy và dừng container; giải thích volume/network; test có thể bắt một lỗi bạn cố ý tạo; pipeline phát hiện lỗi và lưu artifact; bạn nêu được cách xác định phiên bản đang chạy và cách rollback an toàn. Nếu một mục chưa đạt, lặp lại với ứng dụng nhỏ hơn.

Câu hỏi phỏng vấn để tự trả lời

Image khác container thế nào? Vì sao không nên dùng **latest** làm định danh duy nhất cho release? Unit, integration và smoke test phát hiện các lớp lỗi nào? Nếu pipeline xanh nhưng ứng dụng báo 502 sau deploy, bạn kiểm tra những gì? Vì sao secret trong image là vấn đề ngay cả khi container đã bị xóa?

Đọc workflow CI và nguyên tắc quyền

File **lesson/labs/notes-api/ci-example.yml** là mẫu để sao chép vào **.github/workflows/ci.yml** **sau khi** repository được đưa lên GitHub. Workflow chạy khi push hoặc pull request. **permissions: contents: read** giới hạn token mặc định cho job chỉ đọc mã; nó không cần quyền cloud. Job dùng runner Linux, checkout mã, chuẩn bị Python phiên bản ghi rõ rồi chạy cùng bộ test. Nếu bạn đặt app ở đường dẫn khác, phải sửa **-s** trong lệnh test. Đọc workflow trước khi chạy và kiểm tra phiên bản action theo tài liệu GitHub hiện hành vì chúng có thể thay đổi.

Khi xem một run, đọc từ trên xuống: sự kiện nào kích hoạt, commit SHA nào, runner nào, step đầu tiên fail, exit code và log. Nếu checkout fail thì code chưa tới runner; nếu setup Python fail thì test chưa chạy; nếu unittest fail thì xem assertion; nếu workflow không xuất hiện thì kiểm tra vị trí file và trigger. Đừng gọi mọi trường hợp là "test fail".

Quyền CI nên tăng khi có nhu cầu mới và bằng chứng: build/test không cần quyền deploy. Một job deploy cần điều kiện nhánh/môi trường, identity phù hợp và approval nếu rủi ro yêu cầu. Pull request từ bên ngoài là dữ liệu chưa tin cậy. Không đưa secret vào lệnh **echo** hoặc biến mà test có thể in ra. Sau khi test ổn định, thêm bước build image và artifact theo tài liệu nền tảng; lúc đó cần định nghĩa tag/digest để nối commit với image.

Chuỗi điều tra khi bản phát hành thất bại

Giả sử CI xanh và image build thành công, nhưng sau khi Compose chạy người dùng không thể truy cập. Hãy đi theo thứ tự: image đang chạy có đúng version không (**docker compose images** hoặc thông tin container); container có chạy không (**docker compose ps**); log khởi động báo gì; mapping port đúng chưa; health endpoint từ máy chủ có trả lời không; request có tới ứng dụng không. Nếu container chết do không ghi được **/data**, vấn đề là quyền/volume, không phải DNS. Nếu API trả 404 cho **/health**, có thể đang chạy image sai hoặc route code đã đổi.

Viết một báo cáo ngắn: triệu chứng, bằng chứng, giả thuyết, thay đổi đã thử, kết quả và hành động tiếp theo. Báo cáo này làm quá trình debug có thể review. Khi thay đổi một yếu tố và kiểm tra lại, bạn biết

điều gì đã giúp; khi thay mọi thứ cùng lúc, bạn chỉ biết hệ thống bỗng chạy.

Tách các trách nhiệm trong một lần phát hành

Developer tạo commit. CI lấy đúng commit, chạy kiểm tra, đóng gói artifact bất biến, gắn định danh và lưu vào registry. CD chọn artifact đã được kiểm tra, cập nhật cấu hình triển khai và xác nhận dịch vụ hoạt động. GitOps dùng Git làm nơi mô tả trạng thái mong muốn; controller so trạng thái đó với cluster và đồng bộ. Helm tạo bộ manifest Kubernetes từ chart và values. Ansible cấu hình máy chủ hoặc chạy tác vụ có thể lặp. Các công cụ này cùng nằm trong một chuỗi nhưng không thay thế nhau.

Ví dụ Notes API: commit **a1b2c3** vượt unit test; pipeline build image **notes-api@sha256:...**; một thay đổi trong repo môi trường cập nhật image digest; Argo CD phát hiện thay đổi và đồng bộ Deployment; smoke test gọi **/health** và **GET /notes**; monitoring theo dõi error rate. Nếu cùng một tag **latest** bị ghi đè, bạn khó biết chính xác mã nào đã chạy. Vì vậy trong báo cáo release cần lưu commit, digest, môi trường, thời điểm và người/phê duyệt liên quan.

Câu hỏi kiểm tra: Nếu CI xanh nhưng ứng dụng lỗi sau triển khai, CI đã chứng minh được điều gì? Trả lời đúng: chỉ các kiểm tra thực sự được chạy trên artifact/điều kiện cụ thể; nó không chứng minh môi trường production và đường truy cập người dùng đều hoạt động. Cần deployment verification và tín hiệu vận hành.

Cấu trúc pipeline và các cổng chất lượng

Một pipeline thực dụng có các bước: kiểm tra cú pháp/định dạng; unit test; kiểm tra dependency và secret; build image; integration test với dependency thật hoặc giả lập có chủ đích; quét image; publish; triển khai vào staging; smoke/E2E; phê duyệt nếu cần; triển khai production; kiểm tra sau release. Không nhất thiết mọi dự án cần tất cả ở ngày đầu. Thứ tự nên chặn lỗi rẻ và nhanh trước lỗi đắt và chậm. Các bước phải fail khi điều kiện thất bại, không chỉ in cảnh báo.

Unit test kiểm tra một đơn vị logic. Integration test kiểm tra ranh giới như app-database. E2E chạy luồng người dùng. SAST đọc mã; SCA kiểm tra dependency; DAST tương tác với ứng dụng chạy. Kết quả quét cần được phân loại theo khả năng khai thác, bối cảnh và chính sách, thay vì coi số CVE là một điểm duy nhất. Các cổng như coverage threshold không chứng minh chất lượng nếu test chỉ chạy qua dòng mã mà không kiểm tra hành vi.

Artifact phải tách khỏi pipeline chạy sau. Không build lại từ cùng branch ở mỗi môi trường vì dependency hoặc base image có thể đổi. Chuyển **cùng digest** từ staging sang production. Tài khoản CI chỉ cần quyền đọc mã, build/push vào registry và tạo thay đổi triển khai đúng phạm vi; không đặt cloud admin key dài hạn trong biến pipeline nếu có thể dùng danh tính ngắn hạn.

Lab 1 - Vẽ pipeline có thể kiểm chứng

Đầu vào: **lesson/labs/notes-api/** và **ci-example.yml**. Đọc file cấu hình và chỉ ra trigger, job, runner, nơi gọi test và điều kiện fail. Thêm một test cố ý thất bại vào bản sao lab; chạy test local trước rồi so với kết quả pipeline nếu có repository. Ghi lại: commit gây lỗi, bước dừng, log bằng chứng và bản sửa. Sau đó viết ra ba bước chưa có trong ví dụ nhưng cần trước production: image scanning, staging smoke test và cách xác nhận rollback. **Kết quả mong đợi:** pipeline đỏ khi test lỗi; khi sửa lại, pipeline xanh cho đúng commit. **Dọn dẹp:** xóa nhánh/thử nghiệm lỗi, không xóa bằng chứng học tập đã ghi.

Ánh xạ GitLab CI, Jenkins, Azure DevOps và GitHub Actions

Ý niệm	GitLab CI	Jenkins	Azure DevOps	GitHub Actions
Định nghĩa	**.gitlab-ci.yml**	**Jenkinsfile**	**azure-pipelines.yml**	**.github/workflows/*.yml**
Đơn vị chạy	job trong stage	stage/step trên agent	stage/job/step	job/step trên runner
Máy thực thi	runner	agent/node	agent	runner
Bí mật và quyền	CI variables/identity	credentials	service connection/identity	secrets/OIDC
Artifact	job artifact/registry	archive/registry	pipeline artifact/registry	artifact/registry

GitLab CI gắn chặt với merge request, stages và runners. Jenkins rất linh hoạt qua pipeline/plugin nhưng bạn phải vận hành controller/agent, cập nhật plugin, quản trị credentials và backup. Azure DevOps Pipelines dùng YAML, stages, environments và service connections; với Azure cần hiểu danh tính của service connection. GitHub Actions dùng events, jobs, runners, permissions và reusable workflows. Một hệ không tự an toàn chỉ vì YAML trông ngắn. Điểm cần đánh giá là quyền của runner, nguồn action/plugin, kiểm soát branch, cache, artifact và đường phê duyệt.

Tình huống: pipeline chạy trên mã pull request không đáng tin

Một PR từ bên ngoài sửa script build. Nếu runner có secret production, script có thể đọc secret. Cần tách job kiểm tra mã không tin cậy khỏi job có quyền publish/deploy; chỉ cấp quyền tối thiểu cho từng job; dùng protected branches/environments và review trước bước nhạy cảm. Đừng cho job PR fork dùng credential triển khai. Trong phỏng vấn, nêu rõ ranh giới tin cậy và cách kiểm chứng cấu hình thực tế của nền tảng đang dùng.

Bài tập chuyển công cụ

Viết trên giấy pipeline bốn bước test -> build -> push -> deploy. Với mỗi CI trên, xác định cấu hình nằm đâu, runner nào thực thi, artifact đi qua đâu, và ai giữ quyền deploy. Bạn không cần thuộc cú pháp của bốn YAML; cần biết đọc tài liệu và phát hiện chỗ thiếu kiểm soát.

Case study có lời giải: deploy xanh, người dùng vẫn lỗi

Bối cảnh: pipeline CI xanh; Helm release báo deployed; Argo CD báo Synced/Healthy. Sau năm phút, tỷ lệ 5xx ở endpoint tạo ghi chú tăng. Hỏi đầu tiên: smoke test đã gọi endpoint nào, dùng dữ liệu/quyền gì, telemetry gắn phiên bản nào, lỗi chỉ ở Pod mới hay mọi Pod? **/health** đơn giản có thể vẫn 200 trong khi đường ghi DB thất bại. Nếu chỉ version mới lỗi, dừng promotion và rollback image/config theo runbook; nếu migration DB không tương thích, phải xử lý schema trước khi quay app cũ. Nếu mọi version lỗi cùng lúc, điều tra dependency chung như DB, Vault hoặc ingress.

Bài học release: định nghĩa cổng sau deploy theo luồng người dùng quan trọng, theo dõi error/latency trong khoảng đủ dài, có người/on-call nhận cảnh báo và ghi digest đang chạy. **Healthy** của controller là một tín hiệu hạ tầng, không phải xác nhận nghiệp vụ. Cổng CI phải giữ log và test report cho đúng commit để sau sự cố tái dựng được chuỗi phát hành.

Một pipeline tốt phải chứng minh điều gì?

Pipeline không chỉ là danh sách stage **build/test/deploy**. Hãy chọn một commit cụ thể rồi theo dấu: source và dependency nào tạo artifact; test nào đã chạy trên artifact ấy; ai có thể đổi pipeline; ai phê duyệt production; bản chạy hiện tại có đúng artifact được duyệt không. Nếu build lại image ở mỗi môi trường từ cùng tag, bạn có thể có byte khác nhau vì dependency/base image đổi. Build một lần, định danh bằng digest, promote cùng artifact qua staging và production là mô hình dễ truy vết hơn.

Một workflow tối thiểu có **lint/unit**, **integration**, **build**, **scan**, **publish**, **deploy-staging**, **smoke**, **approve-production**, **deploy-production**, **observe**. Không nhất thiết mọi dự án có đủ ngay; mỗi gate cần nói rõ nó giảm rủi ro nào. Unit test bắt logic nhỏ; integration test bắt tương tác DB; SAST tìm pattern mã nguy hiểm nhưng có false positive/negative; SCA báo dependency có CVE nhưng phải triage khả năng ảnh hưởng; DAST kiểm tra ứng dụng đang chạy trong phạm vi test. Không gate nào thay quan sát production.

Artifact, cache và secret là ba thứ khác nhau

Artifact là kết quả cần giữ để dùng ở job sau hoặc audit, ví dụ test report, SBOM, image digest. Cache tăng tốc bước lặp và có thể bỏ/rebuild; không dùng cache làm bằng chứng phiên bản. Secret cấp quyền cho job, phải giới hạn theo branch/environment, thời gian sống và scope. “Masked” chỉ che một số cách hiển thị trong log; code độc hại chạy trong job vẫn có thể gửi secret ra ngoài. Review thay đổi pipeline như review code đặc quyền. GitLab tài liệu hiện hành mô tả protected variables và protected runners cho protected refs; đọc chính sách chi tiết trước khi tin rằng PR từ người lạ không chạm secret.

Lab thiết kế: hai commit chạy chồng nhau

Bối cảnh: commit A và B merge liên tiếp; pipeline A deploy staging chậm, pipeline B deploy staging nhanh hơn. Nếu A hoàn tất sau B, staging có thể quay về bản cũ dù UI pipeline B xanh. Vẽ timeline của hai pipeline và chỉ ra môi trường tại từng phút. **Giải pháp:** serialize deployment vào cùng môi trường bằng cơ chế của nền tảng, hoặc kiểm soát revision mới nhất/guard. Thêm kiểm tra sau deploy rằng digest thực đang chạy bằng digest của pipeline thắng. Không dựa vào thứ tự commit để suy ra thứ tự job hoàn tất.

Bài tập 2: production deploy B, smoke test xanh nhưng error rate tăng sau 10 phút. Tính năng mới tạo queue backlog. Quy trình cần thời gian quan sát, alert theo SLO và quyết định rollback/roll-forward. Nếu DB migration không tương thích A, rollback image đơn giản không an toàn. Ghi quyết định trước release: chỉ số nào kích hoạt dừng, ai được quyết định, lệnh kiểm tra nào, đường phục hồi nào đã tập. Đây là nội dung runbook của release, không thể để tới lúc incident mới nghĩ.

Mapping công cụ qua cùng một khái niệm

GitLab CI dùng **.gitlab-ci.yml**, GitHub Actions dùng workflow YAML, Azure DevOps Pipelines dùng YAML và environments/approvals, Jenkins dùng Jenkinsfile/pipeline. Cú pháp khác nhưng luôn hỏi: trigger, runner/agent, job dependency, artifact handoff, environment, secret scope, approval, concurrency và log audit. Argo CD không build code; nó đồng bộ manifest Git với Kubernetes. Ansible cấu hình máy/ứng dụng theo playbook; Terraform quản lý hạ tầng và state. Kết hợp công cụ theo trách nhiệm thay vì cố bắt một tool làm mọi việc.

Ca phỏng vấn có lời giải

CI xanh nhưng production chạy image cũ. So commit -> image digest publish -> config deploy revision -> sync operation -> image ID trên Pod. Nếu repo triển khai còn digest cũ, lỗi ở promotion. Nếu Git đã đổi nhưng Argo OutOfSync, lỗi ở sync/policy. Nếu Deployment khai báo digest mới nhưng Pod cũ chưa rollout, xem readiness/capacity/rollout status. Câu trả lời “restart Jenkins” không gắn với bằng chứng.

Pipeline có quyền cluster-admin để deploy. Giảm phạm vi: service identity theo môi trường, namespace/resource tối thiểu, token ngắn hạn/OIDC khi hỗ trợ, branch protection và audit. Nếu dùng GitOps, pipeline chỉ cần cập nhật repo deploy qua PR; Argo controller có quyền được giới hạn bằng Project và RBAC cluster. Không có mô hình nào tự động an toàn nếu người được merge manifest có quyền quá rộng.

Đạt bài khi bạn vẽ được chuỗi commit-digest-revision-Pod, chỉ ra một race giữa hai deploy, phân biệt artifact/cache/secret và giải thích hai phương án rollback có điều kiện. Nguồn: GitLab CI pipeline, GitHub Actions, Azure Pipelines YAML, Jenkins Pipeline.

Tự đánh giá

- Giải thích mô hình bằng lời của bạn, không dựa vào tên công cụ.
- Làm lab và lưu kết quả mong đợi cùng kết quả thực tế.
- Tạo một lỗi có kiểm soát, dùng bằng chứng để khoanh vùng và phục hồi.
- Trả lời một câu hỏi thiết kế: tại sao chọn phương án này, tradeoff và giới hạn là gì?

Nguồn chính thức và đọc thêm

- https://docs.gitlab.com/ci/pipelines/
- https://www.jenkins.io/doc/book/pipeline/
- https://learn.microsoft.com/azure/devops/pipelines/yaml-schema/?view=azure-pipelines

- https://docs.github.com/en/actions